Lab - 2: **Experiment No: 2**

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using create_block() with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – is_chain_valid() checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes /mine_block, /get_chain, and /is_valid allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Procedure / Steps

Step 1: Install and Import Required Libraries

Install Flask and import the required Python libraries.

```
!pip install flask==2.2.5
import datetime
import hashlib
import json
from flask import Flask, jsonify
```

Step 2: Implement the Blockchain

The Blockchain class is created to manage blocks, transactions, Proof-of-Work, hashing, and validation.

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        # Create Genesis Block
        self.create_block(
```

```
            proof=1,
            previous_hash='0'
        )
    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'transactions': self.transactions.copy(),
            'previous_hash': previous_hash
        }
        self.chain.append(block)
        return block
    def get_previous_block(self):
        return self.chain[-1]
    def create_Transactions(self, sender, receiver, amount):
        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }
        self.transactions.append(transaction)
        return self.get_previous_block()['index'] + 1
    def proof_of_work(self, previous_proof):
        new_proof = 1
        while True:
            hash_operation = hashlib.sha256(
                str(
                    new_proof ** 2 - previous_proof ** 2
                ).encode()
            ).hexdigest()
            if hash_operation[:3] == '000':
                return new_proof
            new_proof += 1
    def hash(self, block):
        encoded_block = json.dumps(
            block,
            sort_keys=True
        ).encode()
        return hashlib.sha256(
            encoded_block
        ).hexdigest()
    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
```

```
        hash_operation = hashlib.sha256(
            str(
                proof ** 2 - previous_proof ** 2
            ).encode()
        ).hexdigest()
        if hash_operation[:3] != '000':
            return False
        previous_block = block
        block_index += 1
    return True
```

Step 3: Create Blockchain, Add Transaction and Mine Block

Create the blockchain and add a transaction using the student's name and college.

```
blockchain = Blockchain()
block_index = blockchain.create_Transactions(
    sender="Khushi Shukla",
    receiver="VESIT",
    amount=100
)
print("Transaction added successfully!")
print("Transaction will be included in Block:", block_index)
print("\nPending Transactions:")
print(blockchain.transactions)
```

Output:

```
================================
TRANSACTION DETAILS
================================
Transaction added successfully!
Transaction will be included in Block: 2

Pending Transactions:
[{'sender': 'Khushi Shukla', 'receiver': 'VESIT', 'amount': 100}]
```

Now mine the block:

```
previous_block = blockchain.get_previous_block()
previous_proof = previous_block['proof']
print("Mining block...")
proof = blockchain.proof_of_work(previous_proof)
print("Proof found:", proof)
previous_hash = blockchain.hash(previous_block)
new_block = blockchain.create_block(
    proof,
    previous_hash
```

```
)
blockchain.transactions.clear()
print("\nBlock mined successfully!")
print(new_block)
```

Output:

```
============================
MINING BLOCK
============================
Mining block...
Proof found: 533

Block mined successfully!
{'index': 2, 'timestamp': '2026-09-08 09:03:49.472527', 'proof': 533, 'transactions': [{'sender': 'Khushi Shukla', 'receiver': 'VESIT', 'amount': 100}

============================
```

Step 4: Display and Validate the Blockchain

```
print("================================")
print("COMPLETE BLOCKCHAIN")
print("================================")
print(json.dumps(blockchain.chain, indent=4))
print("\n================================")
print("BLOCKCHAIN VALIDITY")
print("================================")
if blockchain.is_chain_valid(blockchain.chain):
    print("All good. The Blockchain is valid.")
else:
    print("The Blockchain is not valid.")
```

Output :

```
================================
COMPLETE BLOCKCHAIN
================================
[
    {
        "index": 1,
        "timestamp": "2026-09-08 09:03:49.470704",
        "proof": 1,
        "transactions": [],
        "previous_hash": "0"
    },
    {
        "index": 2,
        "timestamp": "2026-09-08 09:03:49.472527",
        "proof": 533,
        "transactions": [
            {
                "sender": "Khushi Shukla",
                "receiver": "VESIT",
                "amount": 100
            }
        ],
        "previous_hash": "68f63d9ee6351d063313c9f8dd1518d2b46d192124fd779dc781e42910342af6"
    }
]

================================
BLOCKCHAIN VALIDITY
================================
All good. The Blockchain is valid.
```

Step 5: Implement Flask APIs

Flask is used to provide APIs for mining a block, displaying the blockchain, and checking its validity.

```
app = Flask(__name__)
@app.route('/mine_block', methods=['GET'])
def mine_block():
    if len(blockchain.transactions) == 0:
        return jsonify({
            'message': 'No transactions available.'
        }), 400
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(
        proof,
        previous_hash
    )
    blockchain.transactions.clear()
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'transactions': block['transactions'],
        'previous_hash': block['previous_hash']
    }
    return jsonify(response), 200
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200
@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid = blockchain.is_chain_valid(
        blockchain.chain
    )
    if valid:
        response = {
            'message': 'All good. The Blockchain is valid.'
        }
    else:
        response = {
            'message': 'The Blockchain is not valid.'
        }
    return jsonify(response), 200
```

```
Flask APIs created successfully!
```

```
!pip install pyngrok
```

```
Collecting pyngrok
  Downloading pyngrok-8.1.2-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.13/dist-packages (from pyngrok) (6.0.3)
Downloading pyngrok-8.1.2-py3-none-any.whl (25 kB)
Installing collected packages: pyngrok
Successfully installed pyngrok-8.1.2
```

```
from pyngrok import ngrok

ngrok.set_auth_token("3J3V41ebcSkaNjqfs2joviwSqha_Ch7roY3LXhNSfqqhUKHD")
```

Step 6: Run Flask and Test Using Postman

Since Google Colab was used, pyngrok was used to expose the Flask application.

!pip install pyngrok

Configure ngrok:

```
from pyngrok import ngrok

ngrok.set_auth_token("YOUR_NGROK_AUTH_TOKEN")
```

Run Flask:

```
import threading

def run_flask():
    app.run(
        host='0.0.0.0',
        port=5000,
        debug=False,
        use_reloader=False
    )

thread = threading.Thread(target=run_flask)
thread.start()
```

Create the public URL:

```
import time
from pyngrok import ngrok

time.sleep(3)

public_url = ngrok.connect(5000)

print("Flask URL:")
print(public_url)
```

```
# ==========================================
# START FLASK SERVER
# ==========================================

import threading

def run_flask():
    app.run(
        host='0.0.0.0',
        port=5000,
        debug=False,
        use_reloader=False
    )

thread = threading.Thread(target=run_flask)
thread.start()
```

```
# ==========================================
# CREATE PUBLIC NGROK URL
# ==========================================

import time
from pyngrok import ngrok

time.sleep(3)

public_url = ngrok.connect(5000)

print("Flask Public URL:")
print(public_url)
```

```
Flask Public URL:
NgrokTunnel: "https://rubber-retrace-preface.ngrok-free.dev" -> "http://localhost:5000"
```

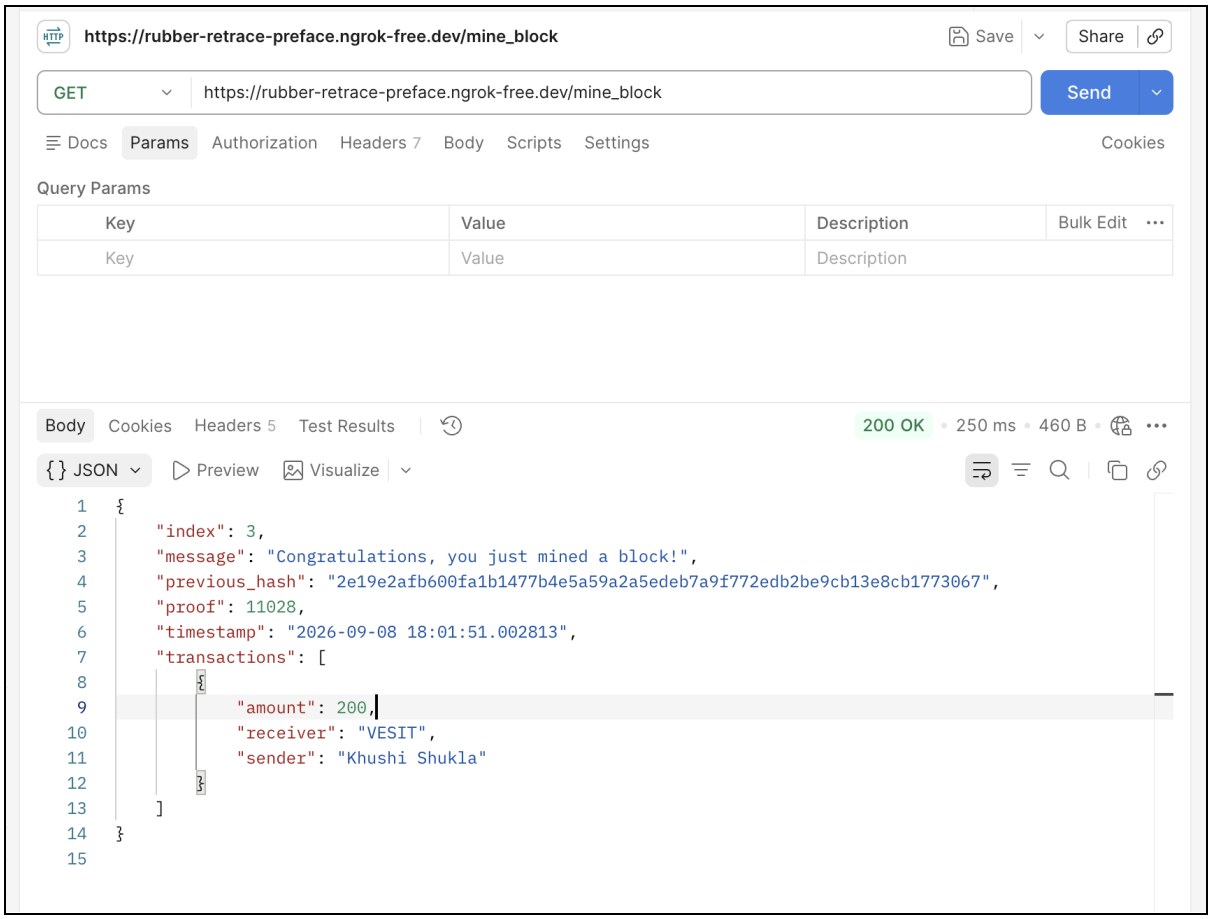
The generated URL was then used in Postman.

API 1 — Mine Block

Method: GET

https://YOUR-NGROK-URL/mine_block -> https://rubber-retrace-preface.ngrok-free.dev/mine_block

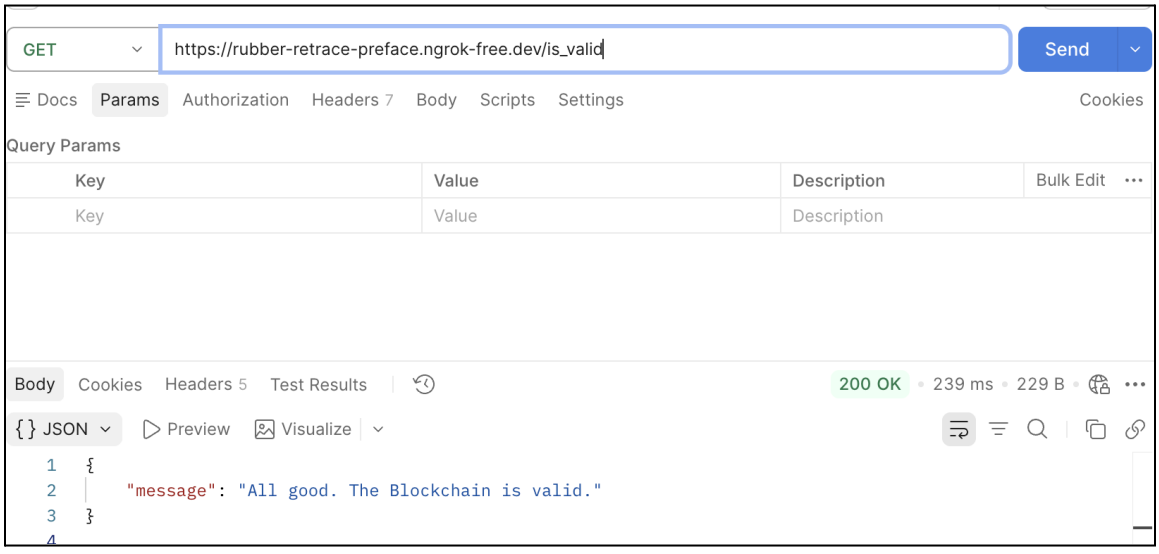
Khushi Shukla_29_D20A



API 2 — Check Validity

Method: GET

https://YOUR-NGROK-URL/is_valid -> https://rubber-retrace-preface.ngrok-free.dev/is_valid



Khushi Shukla_29_D20A

API 3 — Get Blockchain

Method: GET

https://YOUR-NGROK-URL/get_chain -> https://rubber-retrace-preface.ngrok-free.dev/get_chain


```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "0",
            "proof": 1,
            "timestamp": "2026-09-08 17:43:26.758699",
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "327adc8ba14f9c5040854524164ce13981c829deb2160f2a83b857ceb0f52359
            "proof": 533,
            "timestamp": "2026-09-08 17:43:26.759865",
            "transactions": [
                {
                    "amount": 100,
                    "receiver": "VESIT",
                    "sender": "Khushi Shukla"
                }
            ]
        },
        {
            "index": 3,
            "previous_hash": "2e19e2afb600fa1b1477b4e5a59a2a5edeb7a9f772edb2be9cb13e8cb1773067
```


Results / Observations

The experiment successfully demonstrated:

- Creation of a Genesis Block.
- Addition of a transaction from Khushi Singh to VESIT.
- Mining of a new block using Proof-of-Work.
- Use of SHA-256 for block hashing.
- Linking of blocks using the previous block hash.
- Display of the complete blockchain.
- Successful blockchain validation.
- Testing of blockchain operations through Flask and Postman.

Conclusion

The basic blockchain was successfully implemented using Python. Transactions were added and stored in blocks using a Proof-of-Work mechanism with a 000 leading-zero condition. SHA-256 hashing was used to maintain the integrity and connection between blocks. The blockchain was successfully validated, and Flask APIs were used with Postman to mine blocks, display the blockchain, and verify its validity.